

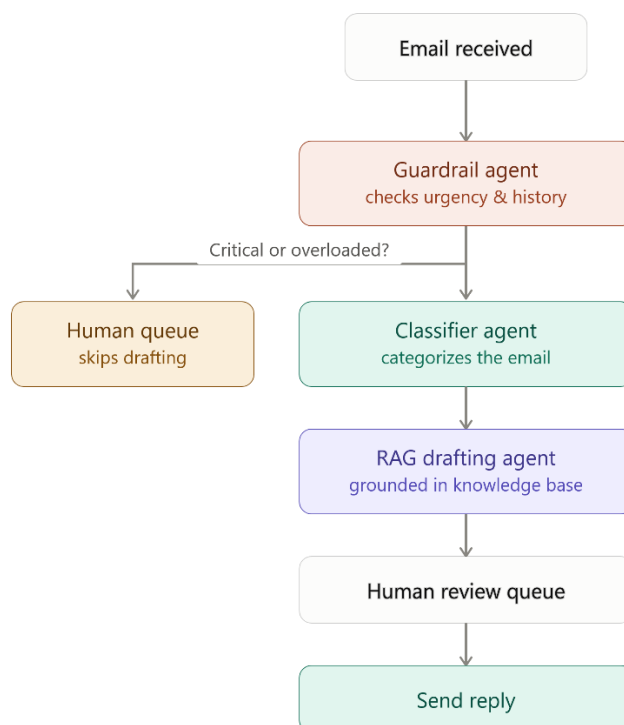
AGENTIC CUSTOMER SUPPORT EMAIL SYSTEM

ARCHITECTURE

1. Overview

The system processes incoming support emails through a pipeline of specialized agents rather than a single monolithic LLM call. This separation makes each stage independently testable, observable, and safe to fail.

2. Pipeline



a) Step 1 : Guardrail/Router Agent (runs first, before drafting)

→ This agent runs first, before any drafting happens. It checks two signals at once. The first is a combination of keyword matching and a lightweight LLM intent check looking for mentions of data loss, service outage, or a security breach, since keyword matching alone is too easy to fool with slightly different wording. The second is a lookup against the customer's contact history to see if they have reached out more than three times in the past seven days. If either condition is true, the email is flagged and routed straight to a human agent, skipping the drafting step entirely.

b) **Step 2 : Classifier Agent**

→ This agent only runs if the email was not flagged in step one. It makes a single LLM call that is constrained to return structured JSON output, including the category, a confidence score, and key entities extracted from the email. Forcing a structured format instead of free text means the rest of the system can branch reliably based on the result.

c) **Step 3 : RAG Drafting Agent**

→ This agent retrieves the most relevant chunks from the internal knowledge base, which is made up of PDFs and FAQs stored in a vector database. The system prompt instructs the model to answer only using the retrieved context and to explicitly refuse to answer rather than guess when the information is not available. This is the main safeguard against hallucinating facts, especially for refund policy questions. A self check step then verifies that the draft only cites facts that actually appear in the retrieved chunks.

d) **Step 4 : Human Review Queue**

→ Every draft passes through human review before anything is sent to the customer. This makes the system slower, but it is a safer choice for a first version of the product.

3. Anti-Hallucination Controls

- (1) RAG grounding — no answer without a source chunk.
- (2) Explicit refusal instruction for unsupported questions.
- (3) Draft self-verification against sources.
- (4) Human-in-the-loop review before send for refund/billing content.

4. Trade-offs

Latency vs. safety: There are three main trade offs in this design. The first is latency versus safety. Adding the guardrail and self check steps means extra LLM calls and extra delay, but they prevent the two most costly failures the system could have, which are missing a genuinely critical issue and inventing facts about the refund policy. The second is structured output versus flexibility. Forcing the classifier to return JSON reduces how freely the model can respond, but it keeps the whole system debuggable and predictable. The third is the human review bottleneck. Requiring every draft to pass through a person adds both latency and cost, but this is an acceptable trade for a first version given the reputational risk of getting something wrong. It can be relaxed later for categories where the system has shown high confidence and low risk.

5. Failure Handling & Observability

Every agent call is logged with its input, output, latency, and token cost, similar to how AgentOps tooling works. LLM calls are wrapped with retries and exponential backoff, and if a call keeps failing, the system defaults to routing the email to a human rather than trying to push a possibly broken response through, meaning it fails safe rather than failing open. When the classifier's confidence score is low, the email is automatically routed to human review instead of letting the system guess. The metrics being tracked include routing accuracy, how often drafts get accepted without edits, average handling time per email, and how often the self check step catches a potential hallucination before it goes out.